

July 11, 2026

```
[235]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[236]: currentversuch = "251"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\251

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\251\Diagramme

Delete UTF8 False encoding

```

[237]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'["][a] '
#     text=re.sub(patterna, "ä", text)
#     patternu=r'["][u] '
#     text=re.sub(patternu, "ü", text)
#     patterno=r'["][o] '
#     text=re.sub(patterno, "ö", text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevocals(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach ` gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'`([a-zA-UO])', convert, text)

```

```

[238]: print(deletevocals("""
Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der
↳ theoretischen Erwartung:
Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen
↳ (das beobachtete Rau-
schen) mit erhöhten Widerständen steige
"""))

```

Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der theoretischen Erwartung: Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen (das beobachtete Rauschen) mit erhöhten Widerständen steige

Runden signifikanter Stellen

```

[239]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher

```

geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.

Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
für Protokolle geeignet sind.

Parameter

****val**** : float
Messwert

****errVal**** : float
Ungenauigkeit des Messwertes

Return

****value_rounded**** : str
Gerundeter Messwert

****error_rounded**** : str
Gerundete Ungenauigkeit des Messwertes

****res**** : str
"Messwert \pm Fehler"
"""

```
einheiten_str = einheiten if einheiten is not None else ""  
if errVal is not None:  
    # Daten zu Dezimal wechseln, da Floats probleme machen  
    val = Decimal(str(val))  
    errVal = Decimal(str(errVal))
```

```
    exp = int(math.floor(math.log10(float(errVal)))) # Die erste   
↳ signifikante Stellenposition wird anhand des errVal bestimmt
```

```
    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn   
↳ 1,2,3 erste signifikante Stelle, dann kommt eine zweite   
↳ Nachkommastellenposition hinzu
```

```
        exp -= 1
```

```
    scale = Decimal(10) ** exp
```

```
    val_round = (val / scale).quantize(Decimal('1'),   
↳ rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so   
↳ verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann   
↳ alle Nachkommastellen weggerundet werden
```

```
    err_round = (errVal / scale).quantize(Decimal('1'),   
↳ rounding=ROUND_CEILING) * scale
```

```

num = f"\\num{{{val_round}}({err_round})}"
qty = f"\\qty{{{val_round}}({err_round})}{{{einheiten_str}}}" if
↪einheiten else num

return float(val_round), float(err_round), num, qty
else:
val = Decimal(str(val))
exp = int(math.floor(math.log10(float(val))))
if round(float(val / (Decimal(10) ** exp))) < 3:      # wenn 1,2,3
↪erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↪hinzu
exp -= 1
scale = Decimal(10) ** exp
val_round = (val / scale).quantize(Decimal('1'),
↪rounding=ROUND_HALF_UP) * scale
num = f"\\num{{{val_round}}}"
qty = f"\\qty{{{val_round}}}{{{einheiten_str}}}" if einheiten else num
return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↪excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↪gewollt sind, die 10e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[240]: def gff(function, errPronePar, latex=True):
        """
        Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

        Parameters
        -----
        **func** : sympy function
            Funktion dessen Fehler bestimmt werden soll.

        **errPronePar** : Array
            Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
            Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
            ↪ungleich den Werten für x, y, z.
            Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
            ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

        Return
        -----
        **absolut_err** : sympy function
        """

```

```

        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
        # wird nach der fehlerbehafteten Variable abgeleitet
        term = sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
        # quadratisch aufsummiert
    errProneParamters.append((variable,Delta))

    absolute_err = sp.simplify(sp.sqrt(error), rational = True)
    relative_err = sp.simplify(sp.sqrt(error/function**2), rational = True)

    if latex:
        # Prints out the Latex Code for the Functions
        print("Funktion:")
        display(Math(sp.latex(function, long_frac_ratio=2)))
        print(sp.latex(function))
        print("Absoluter Fehler:")
        display(Math(sp.latex(absolute_err, long_frac_ratio=2)))
        print(sp.latex(absolute_err))
        print("Relativer Fehler:")
        display(Math(sp.latex(relative_err, long_frac_ratio=2)))
        print(sp.latex(relative_err))
    return function, absolute_err, relative_err, errProneParamters

```

Sigma-Abweichungen

```

[241]: def sigma_abweichung(p1, err_p1, p2, err_p2):
        """
        Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
        Messwert und einem Literaturwert.

```

```

"""
if err_p1 == 0 and err_p2 == 0:
    raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert,
↳ fehlerbehaftet sein!")
else:
    abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
↳ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
↳ Nachkommastellenposition hinzu
    exp -= 1
scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),
↳ rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round),res

```

```

[242]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
↳ val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &
↳ {nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
↳ \\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

```

```

# 2. Der Body (Schleife läuft über die Werte-Arrays)
for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
    err2_array):

    # Deine Berechnungen (bleiben exakt gleich)
    VAL1 = round_to_sigs(val1, err1, r'')[2]
    VAL2 = round_to_sigs(val2, err2, r'')[2]

    abs_val = np.abs(val1 - val2)
    abs_delta = np.sqrt(err1**2 + err2**2)

    if abs_delta != 0:
        SIGMA = sigma_abweichung(val1, err1, val2, err2)[1]
    else:
        SIGMA = 0.0

    ABS = round_to_sigs(abs_val, abs_delta, r'')[2]

    rel = abs_val / np.abs(val1) * 100 if val1 != 0 else 0
    rel_delta = rel * np.sqrt((abs_delta / abs_val)**2 + (err1 / val1)**2)
    if abs_val != 0 and val1 != 0 else 0
    REL = round_to_sigs(rel, rel_delta, r'')[2]

    # Eine saubere Zeile nur mit den Werten für LaTeX
    row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
    table_rows.append(row)

body_str = "\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{nam1}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

[243]:

```

# #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)

```



```

#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
#     else:
#         SIGMA=0.0
#     ABS=round_to_sigs(abs,abs_delta,r')[2]
#     rel=abs/np.abs(val2)*100
#     rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
#     REL=round_to_sigs(rel,rel_delta,r')[2]
#     output = textwrap.dedent(f"""
#         \\begin{{table}}[htb]
#         \\centering
#         \\caption{{{}}}
#         \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
#         \\toprule
#         \\midrule
#         \\bottomrule
#         \\end{{tabularx}}
#         \\label{{table:Vergleich_{{nam1}}}}
#         \\end{{table}}
#     """)
#     print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r')[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r')[2]
    else:
        VAL2=val2

```

```

abs=np.abs(val1-val2)
abs_delta=np.sqrt(err1**2+err2**2)
if abs_delta!=0:
    SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
else:
    SIGMA=0.0
ABS=round_to_sigs(abs,abs_delta,r')[2]
rel=abs/np.abs(val1)*100
rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
REL=round_to_sigs(rel,rel_delta,r')[2]
output = textwrap.dedent(f"""
    \\begin{{table}}[htb]
    \\centering
    \\caption{{{}}}
    \\begin{{tabularx}}{{\\textwidth}}{{ZZZZx}}
    \\toprule
    \\
    ↪{{nam1}}[\\unit{{{einheiten}}}]&{{nam2}}[\\unit{{{einheiten}}}]&\\text{{abs.Abw.
    ↪}}[\\unit{{{einheiten}}}]&\\text{{proz.Abw.
    ↪}}[\\unit{{\\percent}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{{nam1}}}}
    \\end{{table}}
""")
print(output)

```

Zählrohrcharakteristik

```

[244]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.8,6)))
plt.title(r'Zählrohrcharakteristik')
plt.xlabel(r'$U$ [V]')
plt.ylabel(r'Ereignisse N [counts]')

U=np.array([520,550,579,600,630,490,530,480])
Delta_U=10
N=np.array([2639,2741,2669,2731,2913,2622,2669,2283])
Delta_N=np.sqrt(N)

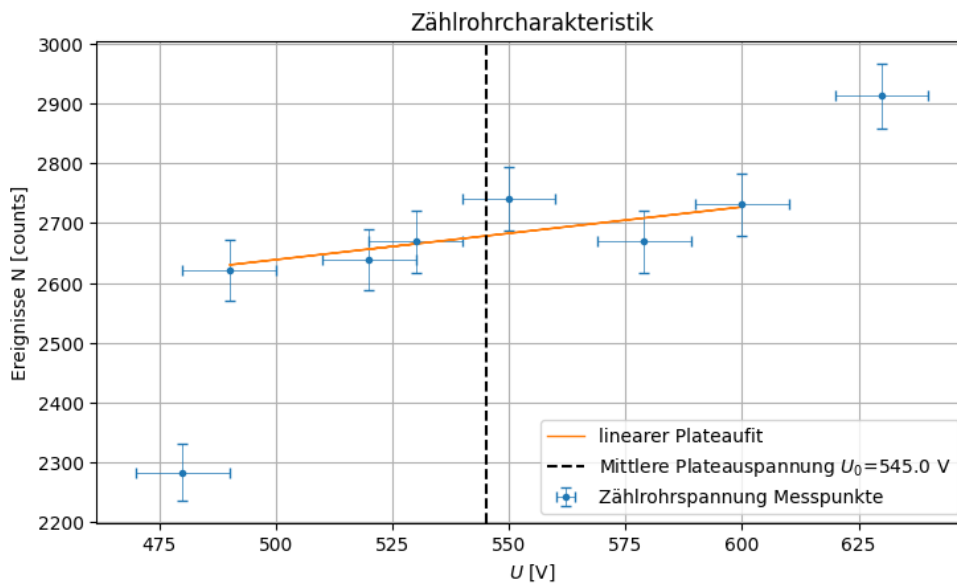
plt.errorbar(U,N,xerr=Delta_U,yerr=Delta_N,capsize=3,elinewidth=0.
    ↪5,label=rf'Zählrohrspannung Messpunkte',fmt='.')
def lin(x,a,b):
    return a*x+b
U=np.delete(U,[-1,-4])
popt, pcov=curve_fit(lin, U,np.delete(N,[-1,-4]))

```

```

plt.plot(U, lin(U, *popt), label='linearer Plateaufit', linewidth=1, linestyle='-')
middle_U = np.min(U) + (np.max(U) - np.min(U)) / 2
plt.axvline(middle_U, linestyle='--', color='black', label=rf'Mittlere_
↳ Plateauspannung  $U_0 = \{middle\_U\}$  V')
plt.legend()
plt.grid()
# plt.tight_layout()
plt.savefig(Ausgabe_path / "Zählrohrcharakteristik.
↳ png", format="png", bbox_inches='tight')
plt.show()

```



Plateauanstieg

```

[245]: N_0_1=7211
Delta_N_0_1=sqrt(N_0_1)
N_0_3=21834
Delta_N_0_3=sqrt(N_0_3)
N_100_1=7353
Delta_N_100_1=sqrt(N_100_1)
N_100_3=22327
Delta_N_100_3=sqrt(N_100_3)

D_1=N_100_1/60-N_0_1/60
Delta_D_1= np.sqrt((Delta_N_0_1**2 + Delta_N_100_1**2)/60**2)
D_1,Delta_D_1,_,_,latexD_1=round_to_sigs(D_1,Delta_D_1,r'\per\s')
print(latexD_1)
print(round_to_sigs(D_1/Delta_D_1,0.01,r'')[0])

```

```

print(sigma_abweichung(N_0_1,Delta_N_0_1,N_100_1,Delta_N_100_1)[1])
rel=D_1/N_0_1*60*100
Delta_rel=rel*np.sqrt(Delta_N_0_1**2/N_0_1**2 + Delta_D_1**2/D_1**2)
rel,Delta_rel,_,latexrel=round_to_sigs(rel,Delta_rel,r'\percent')
print(latexrel)

D_3=N_100_3/180-N_0_3/180
Delta_D_3=np.sqrt((Delta_N_0_3**2 + Delta_N_100_3**2)/180**2)
D_3,Delta_D_3,_,latexD_3=round_to_sigs(D_3,Delta_D_3,r'\per\s')
print(latexD_3)
print(round_to_sigs(D_3/Delta_D_3,0.01,r'')[0])
print(sigma_abweichung(N_0_3,Delta_N_0_3,N_100_3,Delta_N_100_3)[1])
rel=D_3/N_0_3*180*100
Delta_rel=rel*np.sqrt(Delta_N_0_3**2/N_0_3**2 + Delta_D_3**2/D_3**2)
rel,Delta_rel,_,latexrel=round_to_sigs(rel,Delta_rel,r'\percent')
print(latexrel)

N_0_3=21834
N_100_3=22327
n_0=N_0_3/180
Delta_n_0=sqrt(N_0_3)/180
n_100=N_100_3/180
Delta_n_100=sqrt(N_100_3)/180
t=(n_100+n_0)/(0.01**2*(n_100-n_0)**2)
Delta_t= 10000*np.sqrt((Delta_n_0**2*(n_0 + 3*n_100)**2 + Delta_n_100**2*(3*n_0_
↪ + n_100)**2)/(n_0 - n_100)**6)
t,Delta_t,_,latex=round_to_sigs(t,Delta_t,r'\s')
print(latex)
print(300000/60)

variation_68 = (Delta_D_1 / (N_0_1/60)) * 100
variation_95 = (2 * Delta_D_1 / (N_0_1/60)) * 100
print(variation_68)
print(variation_95)
variation_68 = (Delta_D_3 / (N_0_3/180)) * 100
variation_95 = (2*Delta_D_3 / (N_0_3/180)) * 100
print(variation_68)
print(variation_95)

```

```

\qty{2.4(2.1)}{\per\s}
1.143
\num{1.2}
\qty{2.0(1.8)}{\percent}
\qty{2.7(1.2)}{\per\s}
2.25
\num{2.4}
\qty{2.2(1.0)}{\percent}
\qty{300000(300000)}{\s}

```

```

5000.0
1.7473304673415617
3.4946609346831234
0.9892827699917559
1.9785655399835118

```

```

[246]: # Deine Raten und Fehler aus der vorherigen Rechnung
n_0 = 21834 / 180
Delta_n_0 = np.sqrt(21834) / 180

n_100 = 22327 / 180
Delta_n_100 = np.sqrt(22327) / 180

# 1. Absolute Differenz (Effekt der Spannungserhöhung)
D = n_100 - n_0

# 2. Absoluter Fehler der Differenz
Delta_D = np.sqrt(Delta_n_0**2 + Delta_n_100**2)
print(D,Delta_D)
# 3. Prozentuale Variation für 68% (1 Sigma)
variation_68 = (Delta_D / D) * 100

# 4. Prozentuale Variation für 95% (2 Sigma)
variation_95 = (2 * Delta_D / D) * 100

print(f"Variation bei 68% Vertrauensniveau: {variation_68:.1f} %")
print(f"Variation bei 95% Vertrauensniveau: {variation_95:.1f} %")

```

```

2.73888888888888943 1.1674732661438092
Variation bei 68% Vertrauensniveau: 42.6 %
Variation bei 95% Vertrauensniveau: 85.3 %

```

```

[247]: n05,N = np.loadtxt(Messwerte_path/"data_aufgabe2.txt", delimiter="," ,
    ↪ skiprows=4, unpack=True) # Zerfälle pro 0.5s, Anzahl der Ereignisse, bei
    ↪ denen n5 aufgetreten sind
fehlerN=np.sqrt(N)
fehlerN2000=fehlerN[13:-16]
plt.close('all')
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(figsize_cm(14,6)),sharey=True)
fig.suptitle('Statistik des radioaktiven Zerfalls: 2000 Messungen bei Torzeit 0.
    ↪ 5s')

ax1.bar(n05,N,yerr=fehlerN,error_kw={'capsize': 3,'elinewidth': 0.
    ↪ 5},label='Ereignishäufigkeiten')
ax1.set_title('')
ax1.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.5s) [s$^{-1}$]')
ax1.set_ylabel(r'Häufigkeit $N$ des Ereignisses "$n$ Zerfälle")

```

```

ax2.errorbar(n05, N, fehlerN,fmt=".",capsize=3,elinewidth=0.
    ↪5,label='Ereignishäufigkeiten')
ax2.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.5s) [s-1 $]')
def gaussian(x, A, mu, sig): #A: Fläche der Gaussfunktion
    return A/(sqrt(2*pi)*sig)*exp(-(x-mu)**2/2/sig**2)
popt, pcov=curve_fit(gaussian,n05[13:-16], N[13:
    ↪-16],p0=[2000,75,8],sigma=fehlerN[13:-16],absolute_sigma=True)
def poisson(x, A_p, mu_p):
    return A_p*exp(-mu_p)*mu_p**x/gamma(x+1)
popt_p, pcov_p = curve_fit(poisson, n05[13:-16], N[13:-16], p0=[2000,
    ↪75],sigma=fehlerN[13:-16], absolute_sigma=True)
x=np.linspace(30,110, 100)
ax2.plot(x, gaussian(x,*popt), label='Gauss')
ax2.plot(x, poisson(x,*popt_p), label='Poisson',color='red',linestyle=(0, (5,
    ↪10)),linewidth=1)
diff_2000=gaussian(x,*popt)-poisson(x,*popt_p)
x_diff_2000=np.arange(int(n05[13]),int(n05[-16]),1)
diff_2000_g=-N[13:-16]+gaussian(x_diff_2000,*popt)
diff_2000_p=-N[13:-16]+poisson(x_diff_2000,*popt_p)

ax1.legend()
# ax1.grid()
ax2.legend()
# ax2.grid()

plt.savefig(Ausgabe_path/'Histogramm0.5.png',format="png",bbox_inches='tight')
plt.tight_layout()
plt.show()

print("Gaussfit:")
print("A="+round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r'')[3])
print("mu="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\per{s'})[3])
print("sigma="+round_to_sigs(popt[2],np.sqrt(pcov[2][2]),r'\per{s'})[3])
print("Poissonfit:")
print(r"mu_p="+round_to_sigs(popt_p[1],np.sqrt(pcov_p[1][1]),r'\per{s'})[3])
print(r"A_p="+round_to_sigs(popt_p[0],np.sqrt(pcov_p[0][0]),r'')[3])

#Gauss:
chi2_g=np.sum((gaussian(n05[13:-16],*popt)-N[13:-16])**2/fehlerN[13:-16]**2)
dof_g=len(n05[13:-16])-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red_g=chi2_g/dof_g
print("chi2_g=", round_to_sigs(chi2_g)[0])
print("chi2_red_g=",round_to_sigs(chi2_red_g)[0])
#Poisson:
chi2_p=np.sum((poisson(n05[13:-16],*popt_p)
-N[13:-16])**2/fehlerN[13:-16]**2)
dof_p=len(n05[13:-16])-2 #poisson hat nur 2 Parameter

```

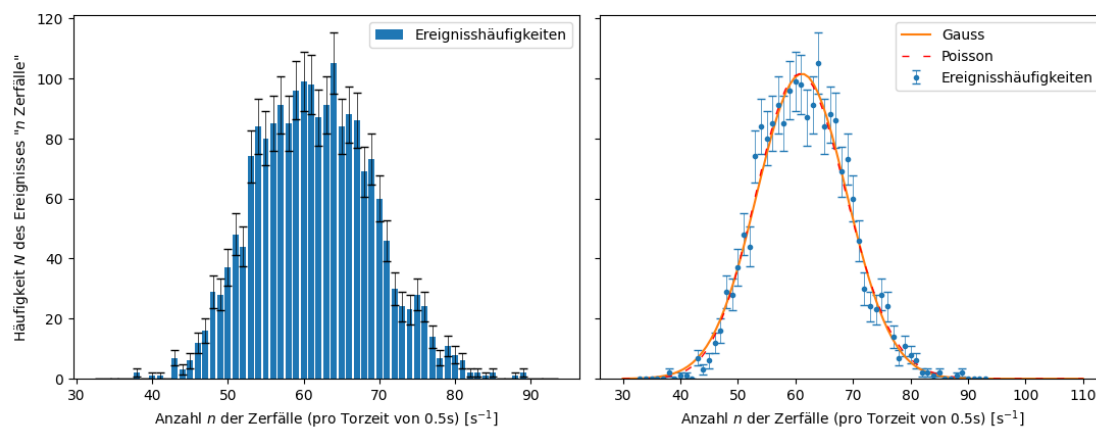
```

chi2_red_p=chi2_p/dof_p
print("chi2_p=", round_to_sigs(chi2_p)[0])
print("chi2_red_p=",round_to_sigs(chi2_red_p)[0])

from scipy.stats import chi2
#Gauss:
prob_g=round(1-chi2.cdf(chi2_g,dof_g),2)*100
#Poisson:
prob_p=round(1-chi2.cdf(chi2_p,dof_p),2)*100
print("Wahrscheinlichkeit Gauss=", prob_g,"%")
print("Wahrscheinlichkeit Poisson=", prob_p,"%")

```

Statistik des radioaktiven Zerfalls: 2000 Messungen bei Torzeit 0.5s



```

Gaussfit:
A=\num{1990(50)}
mu=\qty{61.12(0.21)}{\per\s}
sigma=\qty{7.82(0.19)}{\per\s}
Poissonfit:
mu_p=\qty{61.30(0.21)}{\per\s}
A_p=\num{1990(50)}
chi2_g= 30.0
chi2_red_g= 1.0
chi2_p= 30.0
chi2_red_p= 1.0
Wahrscheinlichkeit Gauss= 40.0 %
Wahrscheinlichkeit Poisson= 52.0 %

```

```

[248]: n05,N = np.loadtxt(Messwerte_path/"data_aufgabe3.txt", delimiter="," ,
    ↳ skiprows=4, unpack=True) # Zerfälle pro 0.5s, Anzahl der Ereignisse, bei
    ↳ denen n5 aufgetreten sind
fehlerN=np.sqrt(N)
fehlerN5000=fehlerN[:-8]

```

```

plt.close('all')
fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(figsize_cm(14,6)))
fig.suptitle('Statistik des radioaktiven Zerfalls: 5000 Messungen bei Torzeit 0.
↳1s')

ax1.bar(n05,N,yerr=fehlerN,error_kw={'capsize': 3,'elinewidth': 0.
↳5},label='Ereignishäufigkeiten')
ax1.set_title('')
ax1.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.1s) [s$^{-1}$]')
ax1.set_ylabel(r'Häufigkeit $N$ des Ereignisses "$n$ Zerfälle")

ax2.errorbar(n05, N, fehlerN,fmt=".",capsize=3,elinewidth=0.
↳5,label='Ereignishäufigkeiten')
ax2.set_xlabel(r'Anzahl $n$ der Zerfälle (pro Torzeit von 0.1s) [s$^{-1}$]')
def gaussian(x, A, mu, sig): #A: Flaeche der Gaussfunktion
    return A/(sqrt(2*pi)*sig)*exp(-(x-mu)**2/2/sig**2)
popt, pcov=curve_fit(gaussian,n05[:-8], N[:-8],p0=[5000,5,2],sigma=fehlerN[:
↳-8],absolute_sigma=True)
def poisson(x, A_p, mu_p):
    return A_p*exp(-mu_p)*mu_p**x/gamma(x+1)
popt_p, pcov_p = curve_fit(poisson, n05[:-8], N[:-8], p0=[5000,
↳5],sigma=fehlerN[:-8], absolute_sigma=True)
x=np.linspace(0,14, 30)
ax2.plot(x, gaussian(x,*popt), label='Gauss')
ax2.plot(x, poisson(x,*popt_p), label='Poisson',color='red',linestyle=(0, (5,
↳10)),linewidth=1)
diff_5000=gaussian(x,*popt)-poisson(x,*popt_p)
x_diff_5000=np.arange(int(n05[0]),int(n05[-8]),1)
diff_5000_g=-N[:-8]+gaussian(x_diff_5000,*popt)
diff_5000_p=-N[:-8]+poisson(x_diff_5000,*popt_p)

ax2.set_yscale('log')
ax1.legend()
# ax1.grid()
ax2.legend()
# ax2.grid()

plt.savefig(Ausgabe_path/'Histogramm0.1.png',format="png",bbox_inches='tight')
plt.tight_layout()
plt.show()

print("Gaussfit:")
print("A="+round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r'')[3])
print("mu="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\per{s}')[3])
print("sigma="+round_to_sigs(popt[2],np.sqrt(pcov[2][2]),r'\per{s}')[3])
print("Poissonfit:")

```



```

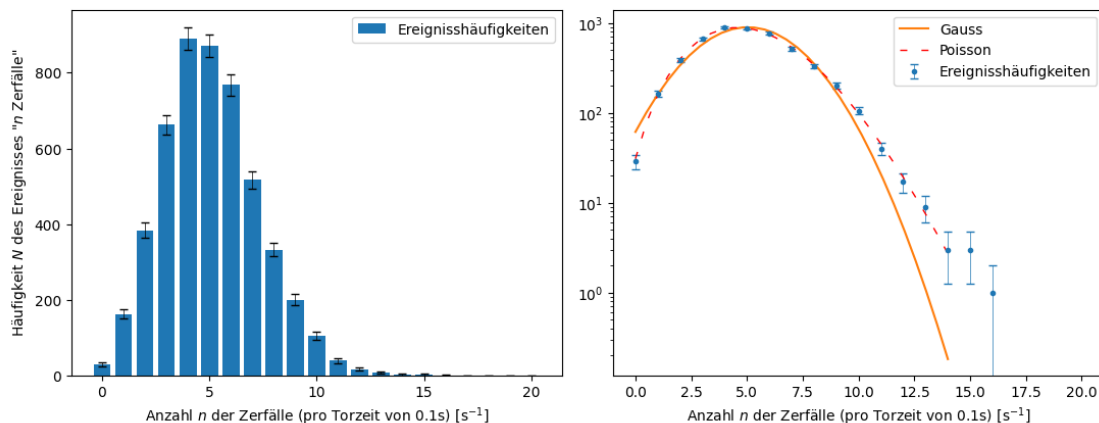
print(r"mu_p="+round_to_sigs(popt_p[1],np.sqrt(pcov_p[1][1]),r'\per\s')[3])
print(r"A_p="+round_to_sigs(popt_p[0],np.sqrt(pcov_p[0][0]),r'')[3])

#Gauss:
chi2_g=np.sum((gaussian(n05[:-8],*popt)-N[:-8])**2/fehlerN[:-8]**2)
dof_g=len(n05[:-8])-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red_g=chi2_g/dof_g
print("chi2_g=", round_to_sigs(chi2_g)[0])
print("chi2_red_g=",round_to_sigs(chi2_red_g)[0])
#Poisson:
chi2_p=np.sum((poisson(n05[:-8],*popt_p)-N[:-8])**2/fehlerN[:-8]**2)
dof_p=len(n05[:-8])-2 #poisson hat nur 2 Parameter
chi2_red_p=chi2_p/dof_p
print("chi2_p=", round_to_sigs(chi2_p)[0])
print("chi2_red_p=",round_to_sigs(chi2_red_p)[0])

from scipy.stats import chi2
#Gauss:
prob_g=round(1-chi2.cdf(chi2_g,dof_g),2)*100
#Poisson:
prob_p=round(1-chi2.cdf(chi2_p,dof_p),2)*100
print("Wahrscheinlichkeit Gauss=", prob_g,"%")
print("Wahrscheinlichkeit Poisson=", prob_p,"%")

```

Statistik des radioaktiven Zerfalls: 5000 Messungen bei Torzeit 0.1s



Gaussfit:

$A = \text{num}\{4900(80)\}$

$\mu = \text{qty}\{5.04(0.04)\} \{\text{per}\s\}$

$\sigma = \text{qty}\{2.174(0.025)\} \{\text{per}\s\}$

Poissonfit:

$\mu_p = \text{qty}\{5.08(0.04)\} \{\text{per}\s\}$

```

A_p=\num{4990(80)}
chi2_g= 120.0
chi2_red_g= 12.0
chi2_p= 6.0
chi2_red_p= 0.6
Wahrscheinlichkeit Gauss= 0.0 %
Wahrscheinlichkeit Poisson= 85.0 %

USELESS

```

```

[268]: plt.close('all')
x_2000=np.linspace(30,110, 100)
# plt.plot(x_2000,diff_2000,label='G-P 2000')
# plt.plot(x,diff_5000,label='G-P 5000')

plt.plot(x_diff_5000,diff_5000_g,label=r'5000: $N_{\text{exp}}-\mu_{\sigma}(n)$')
plt.hlines(np.mean(diff_5000_g),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C0",label='Mean')

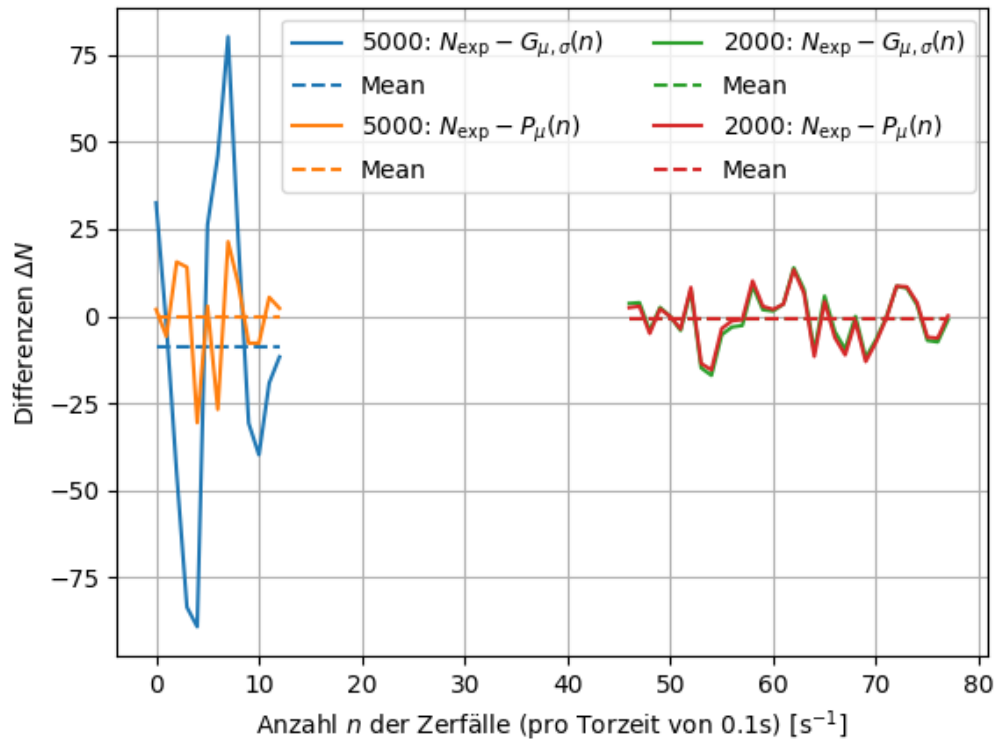
plt.plot(x_diff_5000,diff_5000_p,label=r'5000: $N_{\text{exp}}-P_{\mu}(n)$')
plt.hlines(np.mean(diff_5000_p),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C1",label='Mean')

plt.plot(x_diff_2000,diff_2000_g,label=r'2000: $N_{\text{exp}}-\mu_{\sigma}(n)$')
plt.hlines(np.mean(diff_2000_g),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C2",label='Mean')

plt.plot(x_diff_2000,diff_2000_p,label=r'2000: $N_{\text{exp}}-P_{\mu}(n)$')
plt.hlines(np.mean(diff_2000_p),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C3",label='Mean')

plt.xlabel('Anzahl $n$ der Zerfälle (pro Torzeit von 0.1s) [$s^{-1}$]')
plt.ylabel(r'Differenzen $\Delta N$')
plt.grid()
plt.legend(ncol=2)
plt.savefig(Ausgabe_path/'Abweichungen_dummd.png',format="png",bbox_inches='tight')
plt.show()

```



```
[265]: plt.close('all')
x_2000=np.linspace(30,110, 100)
# plt.plot(x_2000,diff_2000,label='G-P 2000')
# plt.plot(x,diff_5000,label='G-P 5000')

plt.plot(x_diff_5000,(diff_5000_g/fehlerN5000)**2,label=r'5000: $(N_{\text{exp}}-\mu_{\sigma})/\Delta N_{\text{exp}}^2$')
# plt.hlines(np.sum((diff_5000_g/fehlerN5000)**2),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C0",label='Mean')
print(np.sum((diff_5000_g/fehlerN5000)**2))
plt.plot(x_diff_5000,(diff_5000_p/fehlerN5000)**2,label=r'5000: $(N_{\text{exp}}-\mu_{\sigma})/\Delta N_{\text{exp}}^2$')
# plt.hlines(np.sum((diff_5000_p/fehlerN5000)**2),x_diff_5000[0],x_diff_5000[-1],linestyle='--',color="C1",label='Mean')
print(np.sum((diff_5000_p/fehlerN5000)**2))

plt.plot(x_diff_2000,(diff_2000_g/fehlerN2000)**2,label=r'2000: $(N_{\text{exp}}-\mu_{\sigma})/\Delta N_{\text{exp}}^2$')
# plt.hlines(np.sum((diff_2000_g/fehlerN2000)**2),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C2",label='Mean')
```

```

print(np.sum((diff_2000_g/fehlerN2000)**2))

plt.plot(x_diff_2000,(diff_2000_p/fehlerN2000)**2,label=r'2000: $(N_{\text{exp}}-\text{P}(n;\mu,\sigma)/\Delta N_{\text{exp}})^2$')
# plt.hlines(np.sum((diff_2000_p/fehlerN2000)**2),x_diff_2000[0],x_diff_2000[-1],linestyle='--',color="C3",label='Mean')
print(np.sum((diff_2000_p/fehlerN2000)**2))

plt.xlabel('Anzahl $n$ der Zerfälle (pro Torzeit) [s$^{-1}$]')
plt.ylabel(r'$\chi^2_i$')
plt.grid()
plt.legend()
plt.savefig(Ausgabe_path/'Abweichungen.png',format="png",bbox_inches='tight')
plt.show()

```

115.23120822982594
 6.267113003266993
 30.31785092775567
 28.936992832110867

